

UPASS



UPASS Sessions (starting next week)
with **Kelly**

on

- **Tuesdays 3-4pm CB.**
- **Fridays 4-5pm CB.05D.02.020**

www.uts.edu.au/upass-sessions

You can mention what topics you want to
cover next sessions by filling this form:



<https://www.menti.com/al22hkg2o2sa?source=qr-page>



Machine Learning Algorithms and Applications

Lecture 5

(Block 2)

Plan for Today



Feature Engineering



Decision Tree



Multi-Class Classification



Performance Metrics



Lab



Feature Engineering with Pandas

Pandas Data Type

Type	Description	Examples
object	String	"Sydney", "Python"
int64	Integer	8, 2
float64	Decimal	16.8, 0.2
bool	Boolean	True, False
datetime64	Datetime	2011-12-31 2020-04-18
timedelta[ns]	Time difference	3 days 00:15:00 0 days 00:00:14
category	Categorical	"High", "Low"

Data Type Conversion

Casting:

```
df = pd.DataFrame({'col1': [401.0, 202518.0],  
                  'col2': ["High", "Low"]})
```

```
df['col1'] = df['col1'].astype('int64')  
df['col2'] = df['col2'].astype('category')
```

```
df.dtypes
```

```
col1      int64  
col2      category  
dtype: object
```

<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.astype.html>

Each data type has specific attributes and methods that provide more options

Methods for Numeric Type

Casting:

```
df = pd.DataFrame({'coll': [401.0, 202518.0]})
```

```
pd.to_numeric(df['coll'])
```

```
0      401.0  
1    202518.0  
Name: coll, dtype: float64
```

Comparison:

```
df['coll'] > 500
```

```
0    False  
1     True  
Name: coll, dtype: bool
```

https://pandas.pydata.org/pandas-docs/stable/reference/general_functions.html

Methods for Numeric Type

Binnings:

```
df = pd.DataFrame({'age': [15, 28, 45, 68]})
```

```
bins_cut = [0, 20, 30, 40, 50, 60, 100]  
bins_label = ['child', '20s', '30s', '40s', '50s', '60+']  
pd.cut(df['age'], bins=bins_cut, labels=bins_label)
```

```
0    child  
1     20s  
2     40s  
3     60+  
Name: age, dtype: category  
Categories (6, object): [child < 20s < 30s < 40s < 50s < 60+]
```

Mathematical Operation:

```
df['age'].min()
```

15

```
df['age'].max()
```

68

```
df['age'].mean()
```

39.0

```
df['age'].std()
```

22.90560339014597

<https://pandas.pydata.org/pandas-docs/stable/reference/series.html#computations-descriptive-stats>

Methods for DateTime Type

Casting:

```
df = pd.DataFrame({'coll': ["2020-04-01", "2020-05-18"]})  
df.dtypes
```

```
coll    object  
dtype: object
```

```
df['coll'] = pd.to_datetime(df['coll'])  
df.dtypes
```

```
coll    datetime64[ns]  
dtype: object
```

Date Component:

```
df = pd.DataFrame({'coll': ["2020-04-01", "2020-05-18"]})
```

```
df['coll'].dt.month
```

```
0    4  
1    5  
Name: coll, dtype: int64
```

```
df['coll'].dt.dayofweek
```

```
0    2  
1    0  
Name: coll, dtype: int64
```

<https://pandas.pydata.org/pandas-docs/stable/reference/series.html#api-series-dt>

Methods for DateTime Type

Date Difference:

```
df = pd.DataFrame({'end': ["2020-04-01", "2020-05-18"],  
                  'start': ["2018-08-01", "2020-01-31"]})  
df['end'] = pd.to_datetime(df['end'])  
df['start'] = pd.to_datetime(df['start'])
```

```
df['end'] - df['start']
```

```
0    609 days  
1    108 days  
dtype: timedelta64[ns]
```

Date Offset:

```
df['end'] + pd.Timedelta('10 day')
```

```
0    2020-04-11  
1    2020-05-28  
Name: end, dtype: datetime64[ns]
```

```
df['end'].apply(lambda x: x + pd.offsets.BDay(10))
```

```
0    2020-04-15  
1    2020-06-01  
Name: end, dtype: datetime64[ns]
```

<https://pandas.pydata.org/pandas-docs/stable/reference/series.html#api-series-dt>

Methods for Text Type

Text Starting:

```
df = pd.DataFrame({'coll': ["Wooden Table", "Wooden Desk",  
                             "Plastic Table", "Leather Lounge"]})
```

```
df['coll'].str.startswith('Wooden')
```

```
0      True  
1      True  
2     False  
Name: coll, dtype: bool
```

Text Ending:

```
df['coll'].str.endswith('Table')
```

```
0      True  
1      True  
2     False  
Name: coll, dtype: bool
```

<https://pandas.pydata.org/pandas-docs/stable/reference/series.html#api-series-str>

Methods for Text Type

Text Containing:

```
df['coll'].str.contains('Wood')
```

```
0      True
1      True
2     False
Name: coll, dtype: bool
```

Substitution:

```
df['coll'].str.replace('Table', 'Chair')
```

```
0      Wooden Chair
1  Wooden DeskPlastic Chair
2      Leather Lounge
Name: coll, dtype: object
```

<https://pandas.pydata.org/pandas-docs/stable/reference/series.html#api-series-str>



Multi-Class Classification Part 1

What is a Multi-Class Classification Task



Supervised learning

Requires a target variable containing the true values.



Discrete Variable

The target variable can only be a specified set of possible values



Objective

Find a model that will approximate the correlation between the target and features

Multi-Class Classification Application



(Media) Predict main topics of news article: sport, economics, world, ...



(Retail) Predict predict rating: 0, 1, 2, 3 or 4



(Finance) Predict risk-level for a lead: low-risk, medium-risk, high-risk



(Medical) Predict medical condition: none, benign or malignant

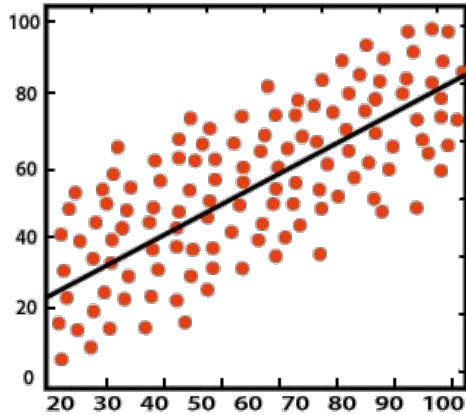
...

Multi-Class Classification Example

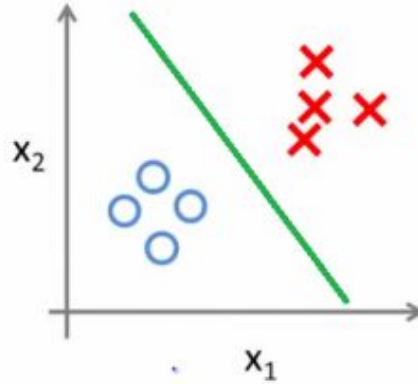
**Predicting Medical
Condition Severity**

Severity	Age	Weight	Height	Blood Pressure	Cholesterol
None	24	70	1.67	96	163
Benign	21	95	1.73	105	134
None	42	87	1.87	116	150
Malignant	38	120	1.95	113	175

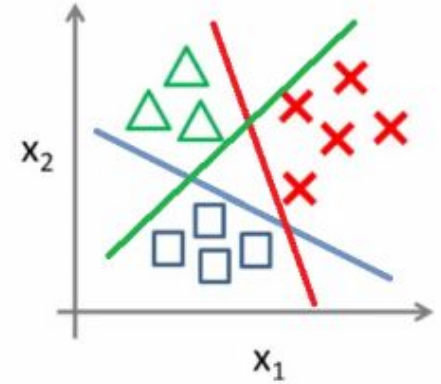
Supervised Learning



Regression



Binary Classification

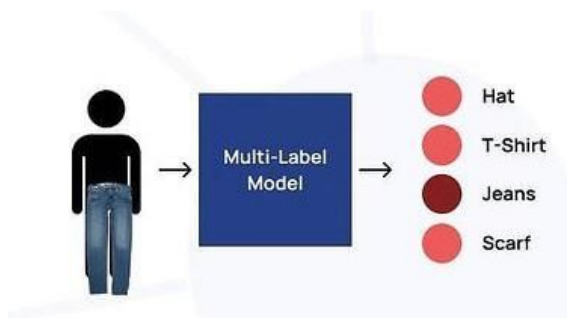


Multi-class Classification

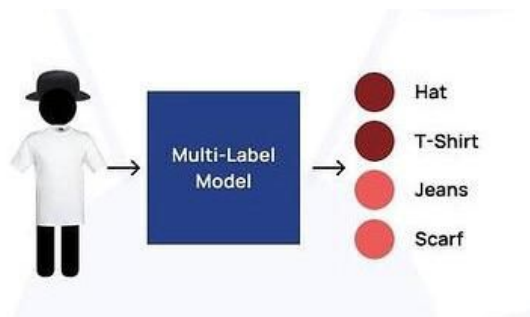


Multi-Label Classification

- Multi-Class: A single observation has only one output class at a time



- Multi-Label: A single observation has multiple output classes at a time





Decision Tree

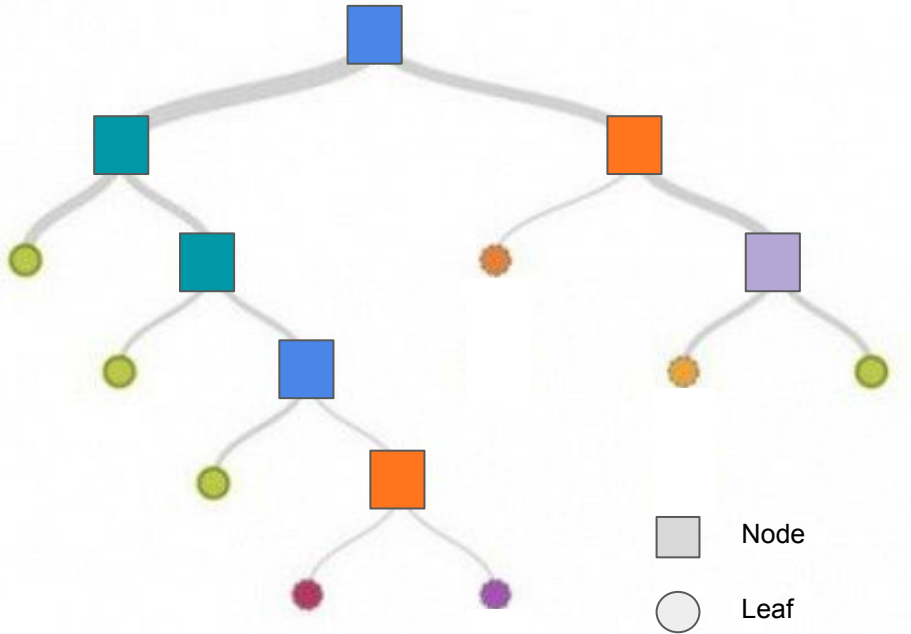
Decision Tree

Objective:

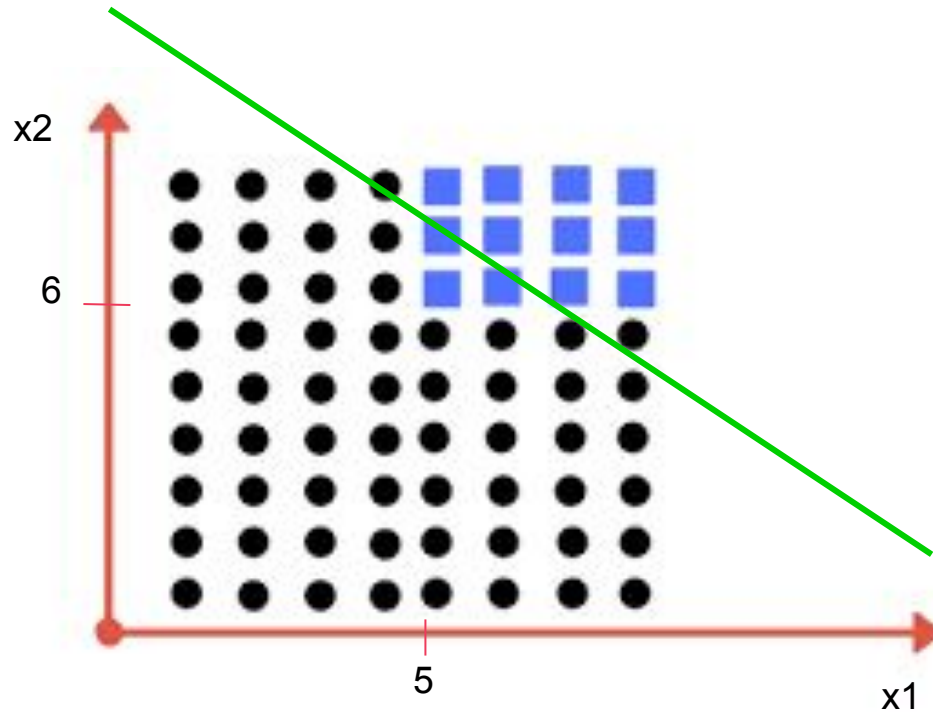
- Split the data into smaller groups of observations that belongs to a specific class using a network of question-answer (condition)

Node: Condition (using a single feature) that splits data into 2 different groups

Leaf: End node (no more split) used for prediction

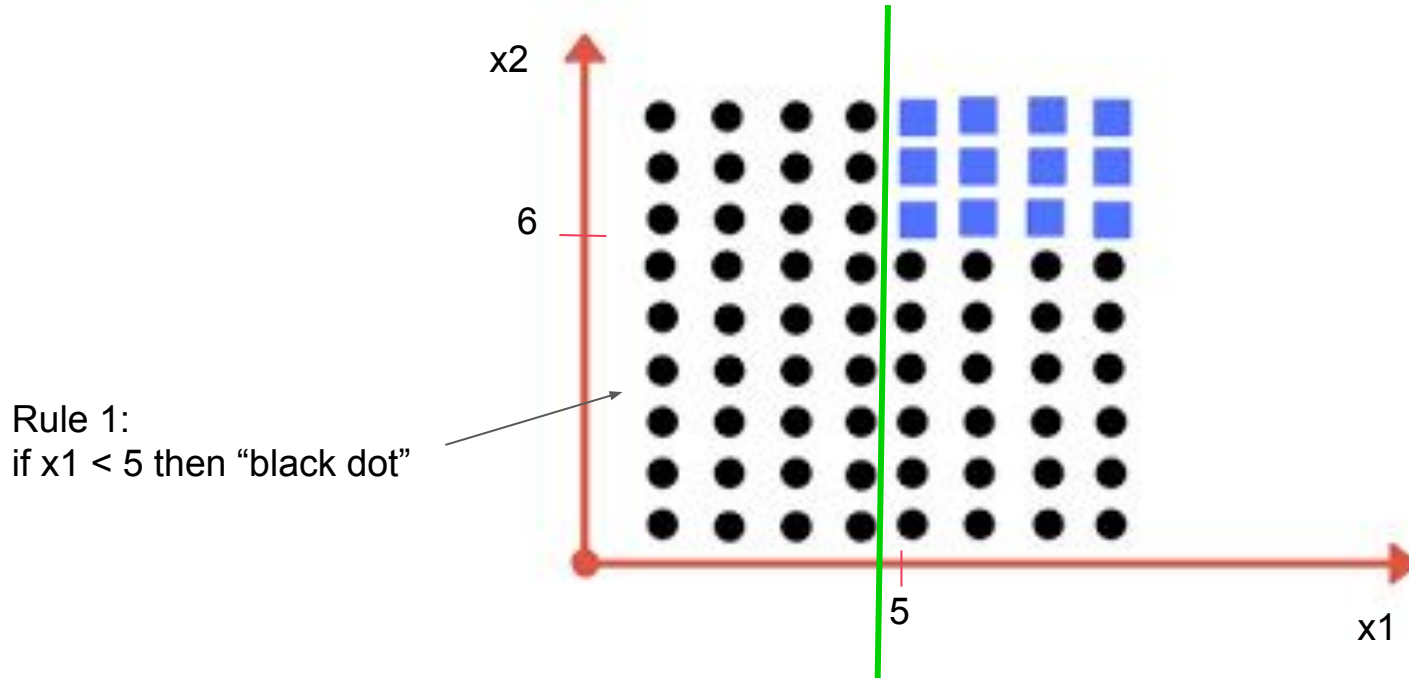


Linear versus Non-Linear

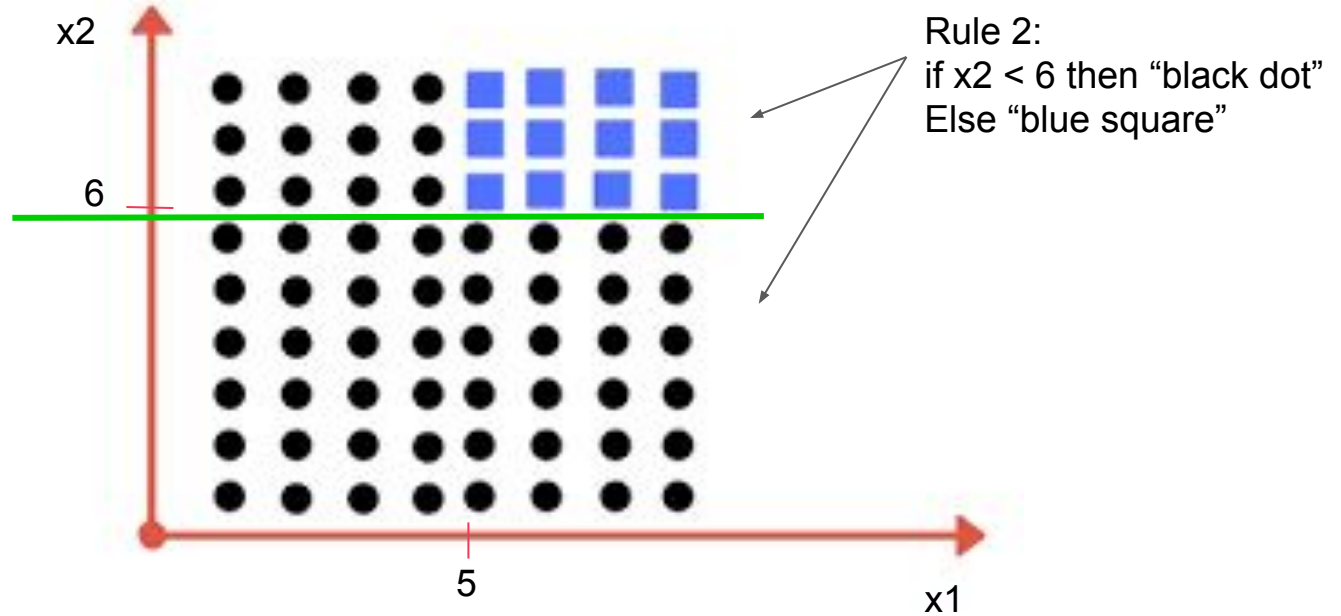


A linear function will not be able to separate perfectly these 2 classes

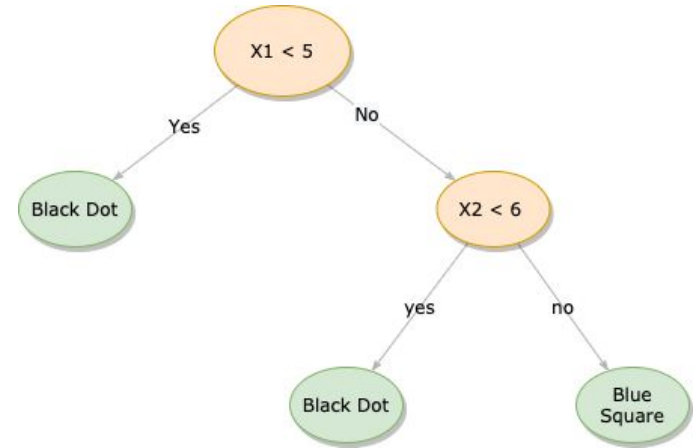
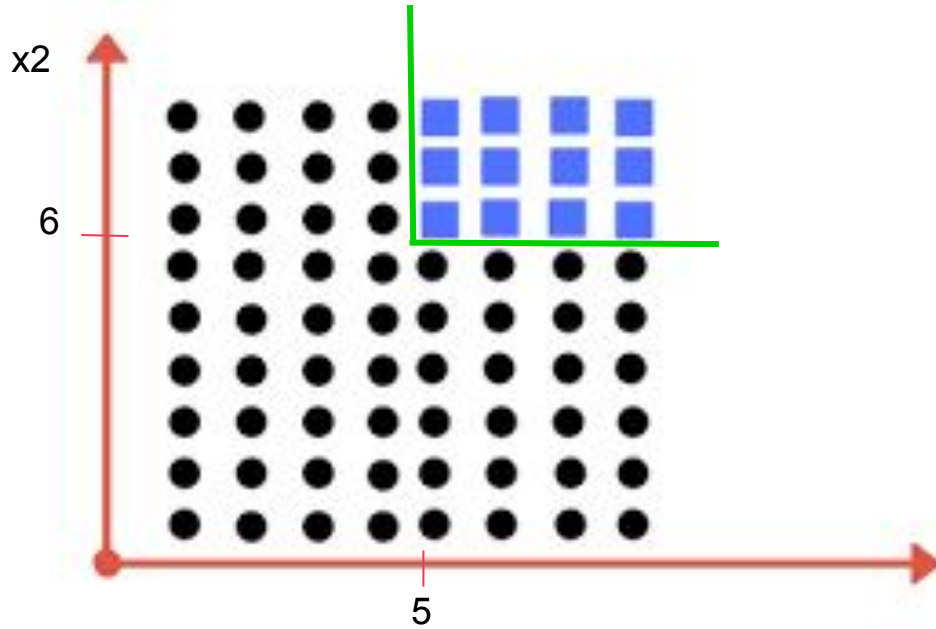
Decision Tree Non-Linearity



Decision Tree Non-Linearity



Decision Tree Non-Linearity



Hyperparameter: Split Criterion

Objective:

- At each node, split the data into 2 groups as uniform as possible (high “purity”)

Impurity Criterion

Gini Index

$$I_G = 1 - \sum_{j=1}^c p_j^2$$

p_j : proportion of the samples that belongs to class c for a particular node

Entropy

$$I_H = - \sum_{j=1}^c p_j \log_2(p_j)$$

p_j : proportion of the samples that belongs to class c for a particular node.

*This is the the definition of entropy for all non-empty classes ($p \neq 0$). The entropy is 0 if all samples at a node belong to the same class.

Hyperparameter: Max Depth

Objective:

- Limit the depth of the tree (ie number of nodes) in order to prevent overfitting



Depth = 1



Depth = 2



Depth = 3



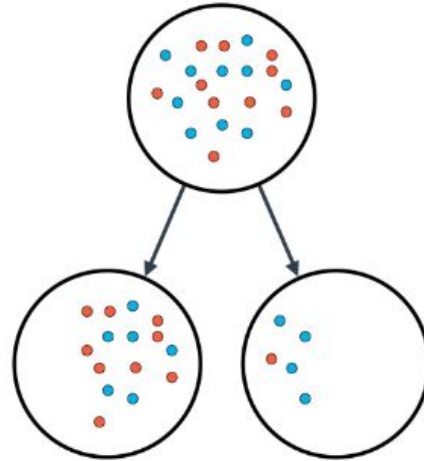
Depth = 4

Hyperparameter: Min Samples Split

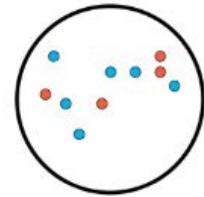
Objective:

- Limit the split into very small groups of observations in order to prevent overfitting

Minimum number of samples to split = 11



No split!



No split!

Decision Tree Classifier

Generic API for sklearn models:



Import class

```
from sklearn.tree import DecisionTreeClassifier
```



Instantiate a model with hyperparameter

```
tree = DecisionTreeClassifier(max_depth=6)
```



Fit a model

```
tree.fit(X, y)
```



Predict with a model

```
tree.predict(new_X)
```

Sklearn Parameters

<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html?highlight=decision%20tree#sklearn.tree.DecisionTreeClassifier>

Parameters: **criterion : {"gini", "entropy"}, default="gini"**

The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "entropy" for the information gain.

splitter : {"best", "random"}, default="best"

The strategy used to choose the split at each node. Supported strategies are "best" to choose the best split and "random" to choose the best random split.

max_depth : int, default=None

The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.

min_samples_split : int or float, default=2

The minimum number of samples required to split an internal node:

- If int, then consider min_samples_split as the minimum number.
- If float, then min_samples_split is a fraction and $\text{ceil}(\text{min_samples_split} * \text{n_samples})$ are the minimum number of samples for each split.

Changed in version 0.18: Added float values for fractions.

min_samples_leaf : int or float, default=1

The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least min_samples_leaf training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression.

The background of the slide is a cosmic scene featuring a vibrant nebula with orange, red, and blue hues. A diagonal line splits the image, with the upper-left portion showing the colorful nebula and the lower-right portion being a solid black space. The title text is centered over the nebula portion.

Multi-Class Classification Metrics

Definitions

Predictions

		Predictions	
		Class 1	Class 0
Actuals	Class 1	True Positive	False Negative
	Class 0	False Positive	True Negative

Precision

Formula:

$$\textit{precision} = \frac{TP}{TP+FP}$$

Precision = 1 (Perfect score)

Higher precision means model is making more True Positive predictions than False Positive

	Pred Class 1	Pred Class 0
Actual Class 1	TP=1	FN=9
Actual Class 0	FP=10	TN=980

$$\textit{precision} = \frac{1}{1+10} = 0.09$$

Recall

Formula:

$$\text{recall} = \frac{TP}{TP+FN}$$

Recall = 1 (Perfect score)

Higher recall means model is making more True Positive predictions than False Negative

	Pred Class 1	Pred Class 0
Actual Class 1	TP=1	FN=9
Actual Class 0	FP=10	TN=980

$$\text{recall} = \frac{1}{1+9} = 0.1$$

F1 score

Formula:

$$F1\ score = 2 * \frac{Precision * Recall}{Precision + Recall}$$

	Pred Class 1	Pred Class 0
Actual Class 1	TP=1	FN=9
Actual Class 0	FP=10	TN=980

$$F1\ score =$$

$$2 * \frac{0.09 * 0.1}{0.09 + 0.1} = 2 * \frac{0.009}{0.19} = 0.095$$

Precision, Recall and F1 score

```
from sklearn.metrics import precision_score
```

```
y_pred = [1, 0, 1, 1]  
y_true = [0, 0, 1, 1]
```

```
precision_score(y_true, y_pred)
```

```
0.6666666666666666
```

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_score.html?highlight=precision#sklearn.metrics.precision_score

```
from sklearn.metrics import recall_score
```

```
y_pred = [1, 0, 1, 1]  
y_true = [0, 0, 1, 1]
```

```
recall_score(y_true, y_pred)
```

```
1.0
```

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.recall_score.html?highlight=recall#sklearn.metrics.recall_score

```
from sklearn.metrics import f1_score
```

```
y_pred = [1, 0, 1, 1]  
y_true = [0, 0, 1, 1]
```

```
f1_score(y_true, y_pred)
```

```
0.8
```

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html?highlight=f1#sklearn.metrics.f1_score

Confusion Matrix

Predictions

Actuals

	Class A	Class B	Class C	Class D	Class E
Class A	980	21	0	54	12
Class B	12	829	23	6	65
Class C	12	15	832	0	21
Class D	24	43	0	243	16
Class E	14	0	18	8	422

Tool used for interpretation. See where your models is performing well or poorly

Confusion Matrix

```
from sklearn.metrics import confusion_matrix
```

```
y_true = ["high", "low", "high", "high", "low", "medium"]  
y_pred = ["low", "low", "high", "high", "low", "high"]
```

```
confusion_matrix(y_true, y_pred, labels=["low", "medium", "high"])
```

```
array([[2, 0, 0],  
       [0, 0, 1],  
       [1, 0, 2]])
```

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html?highlight=confusion%20matrix#sklearn.metrics.confusion_matrix



Assignment

Assignment 2

Objective:

You will have to build a model that can predict if university students are more likely to perform well in the current semester. The results of your model can be used for offering additional support to the students in need by the university.

This assignment requires you to run multiple experiments with the following instructions:

- **Experiment 0:** Perform EDA, data preparation, feature selection, feature engineering, assess baseline model performance.
- **Experiment 1:** Define an hypothesis and approaches (data preparation, feature selection, feature engineering, algorithm, hyperparameter tuning) to test and experiment on.
- **Experiment 2:** Define an hypothesis and approaches (data preparation, feature selection, feature engineering, algorithm, hyperparameter tuning) to test and experiment on.
- **Experiment 3:** Define an hypothesis and approaches (data preparation, feature selection, feature engineering, algorithm, hyperparameter tuning) to test and experiment on.
- **Experiment 4:** Define an hypothesis and approaches (data preparation, feature selection, feature engineering, algorithm, hyperparameter tuning) to test and experiment on.

Assignment 2

Due Date: 25/04/2025 11:59pm

Submission Requirements:

You will need to submit the following files on Canvas:

- 5 Jupyter notebooks (one for each experiment) as **.ipynb** files
- 5 Jupyter notebooks (one for each experiment) as **.pdf** files
- 1 final report and submit it on Canvas as **.pdf** file
- 1 .zip file containing all your files

In total you should submit 12 files on Canvas.

Note: Be careful with the naming convention

More information can be found on Canvas:

<https://canvas.uts.edu.au/courses/34235/assignments/209082>

Assessment Criteria:

	Criteria	Weight (%)
1.	Quality, relevance and cleanliness of code and visualisation	20
2.	Pertinence and quality of scientific experimentation and analytic investigation	20
3.	Depth of discussion of ethics/privacy issues (including matters related to Indigenous people), value, benefits, risks and recommendation for business stakeholders and final users	15
4.	Strength of justification and explanation of models selected, data transformation performed, hyperparameters selected and accuracy of results with evidence supporting claims	25
5.	Clarity and quality of written report, data visualisation and appropriateness of communication style to audience	20



Time for Lab

